

INDEX

Practical No.	Practicals	Page No.	Sign
1.	Install, configure and run Hadoop and HDFS and explore HDFS.	2	
2.	Implement an application that stores big data in Hbase / MongoDB and manipulate it using R / Python	8	
3.	Implement Regression Model Import a data from web storage. Name the dataset and now do Logistic Regression to find out relation between variables. Also check the model is fit or not.	13	
4.	Apply multiple regressions, if data have a continuous independent variable. Apply on above dataset.	15	
5.	Build a classification model (Logistic Regression) a) Install relevant package for classification. b) Choose classifier for classification problem. c) Evaluate the performance of classifier.	18	
6.	Build a Clustering Model (K-means, Agglomerative) a) Clustering algorithms for unsupervised classification. b) Plot the cluster data using R visualizations.	21	
7.	Implement SVM classification techniques	24	
8.	Implement Decision tree classification techniques	27	
9.	Naive Bayes Implementation	29	

Practical 1

Aim: Install, configure and run Hadoop and HDFS and explore HDFS.

Theory:

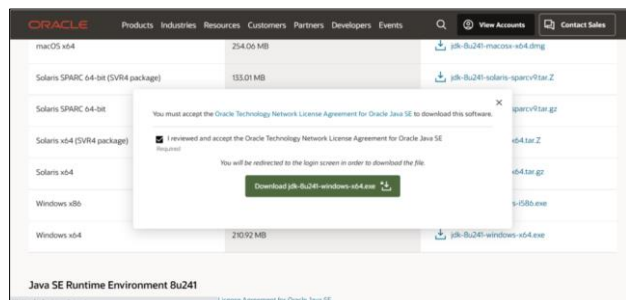
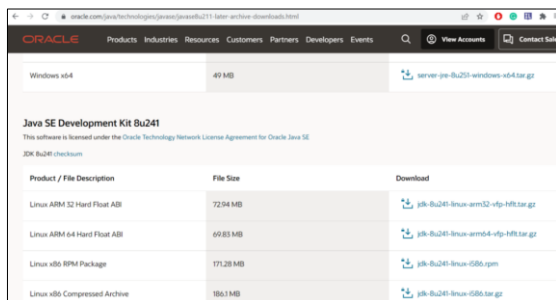
Hadoop is an open-source framework based on Java that manages the storage and processing of large amounts of data for applications. Hadoop uses distributed storage and parallel processing to handle big data and analytics jobs, breaking workloads down into smaller workloads that can be run at the same time. Hadoop Distributed File System (HDFS for short) is the primary data storage system used by Apache Hadoop applications to manage large amounts of data and support related big data analytics applications. HDFS operates as a distributed file system designed to run on commodity hardware. HDFS is fault-tolerant and designed to be deployed on low-cost, commodity hardware.

Features of HDFS are:

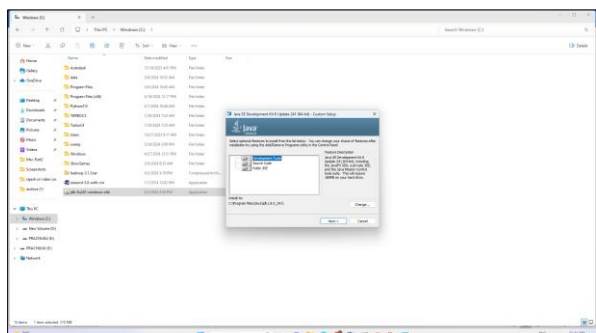
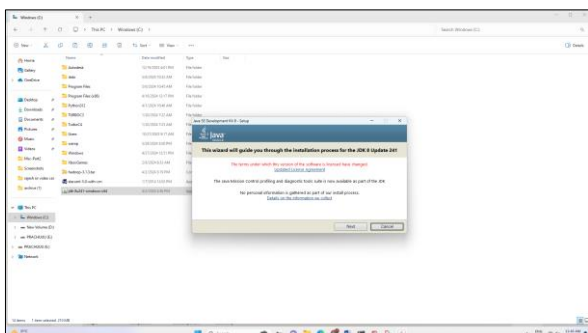
- Data replication. Data replication ensures that the data is always available and prevents data loss.
- Fault tolerance and reliability.
- High availability.
- Scalability.
- High throughput.
- Data locality.
- Snapshots.

Steps: To download and install Hadoop:

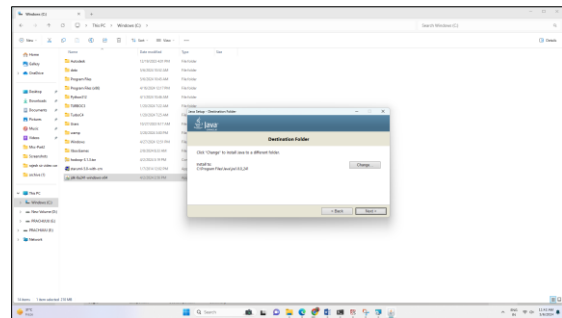
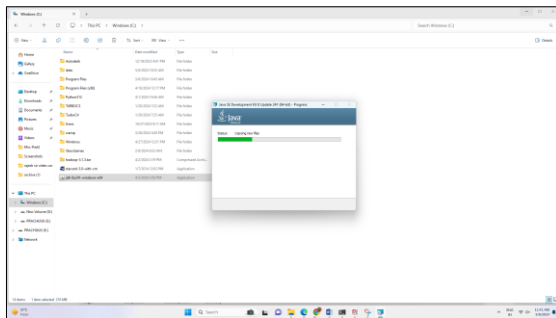
1. Download Java from oracle website. Search for java SE development kit 8 download (8u214)



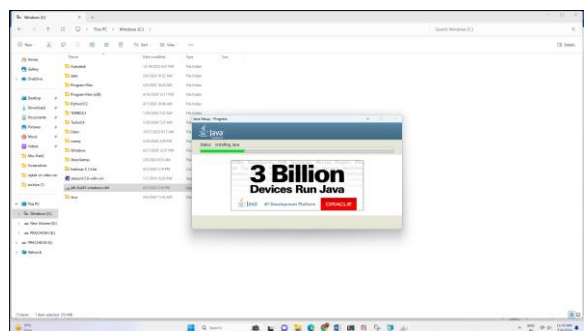
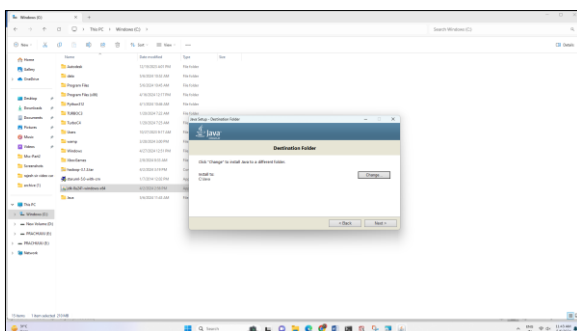
2. To install Java. Double click on the exe file downloaded. A download wizard appears click on next.



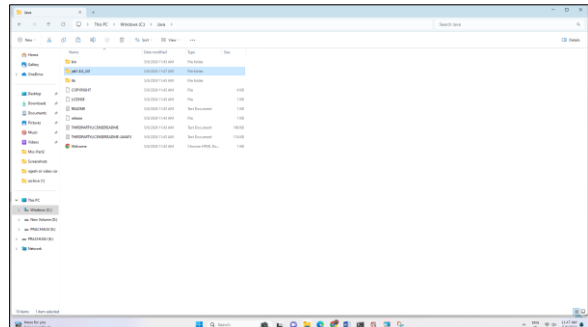
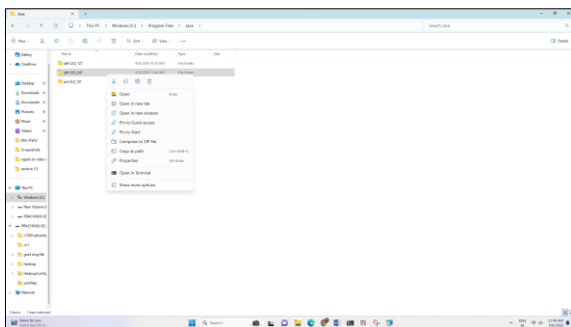
3. The installation is started. Change the destination folder to desired location and click on install.



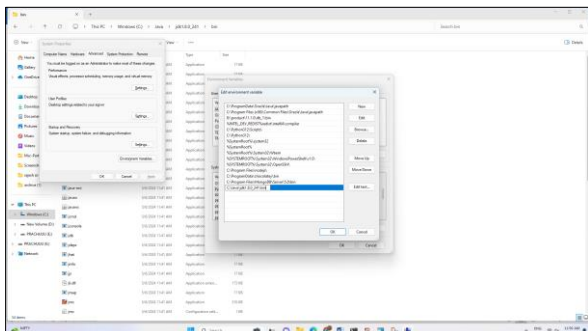
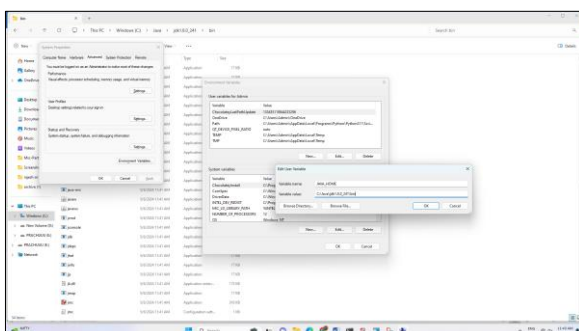
4. Create “Java” folder under C: drive and choose that folder as the destination folder and click on install.



5. Once installation is completed close the wizard and go to the jdk folder installed in the program files. Copy the “jdk1.8.0_241” folder from program files and paste it to the destination folder created earlier (C:/Java).

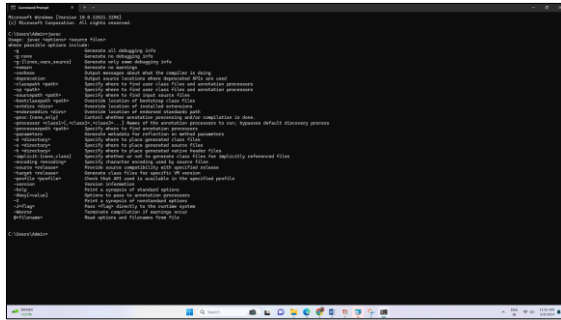


6. Set the Environment variable and system path to this folder for java.

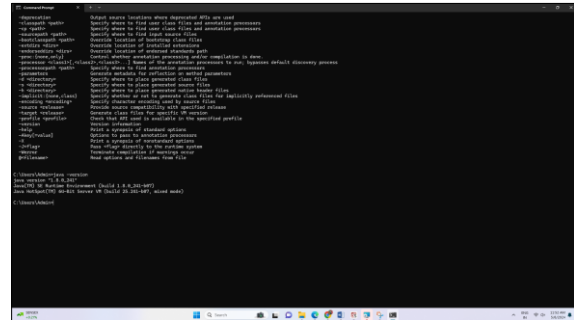


7. Run below commands in cmd to test if the java is installed successfully.

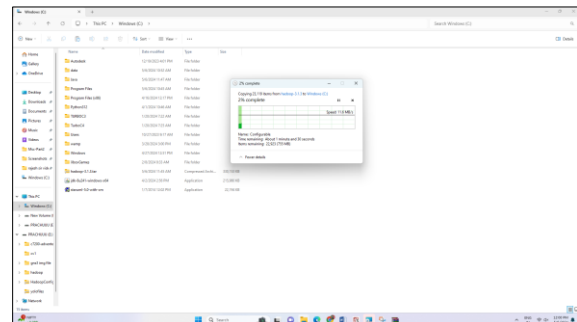
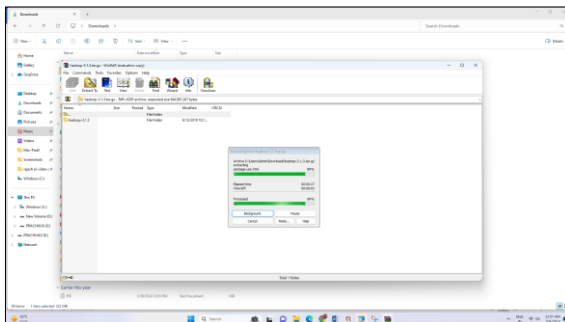
> javac



> java -version



8. Download Hadoop 3.1.3 tar file from web and unzip the tar file once download is completed. Create a folder “hadoop” in C: drive and past all the file content extracted from the tar file.



9. Once the directory is setup, configure below four files.

a. Core-site.xml

```
<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://localhost:9000</value>
  </property>
</configuration>
```

b. Hdfs-site.xml

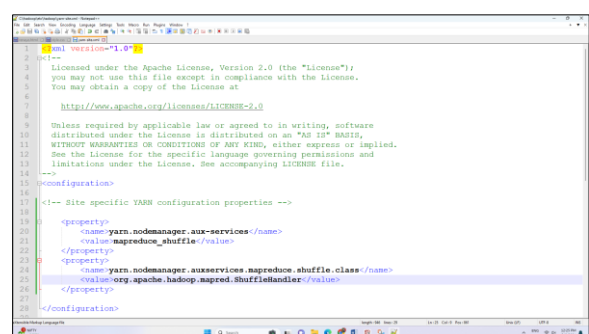
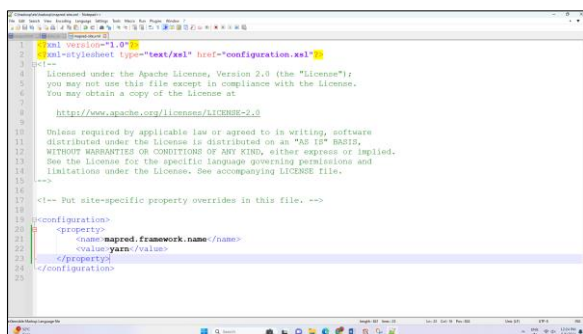
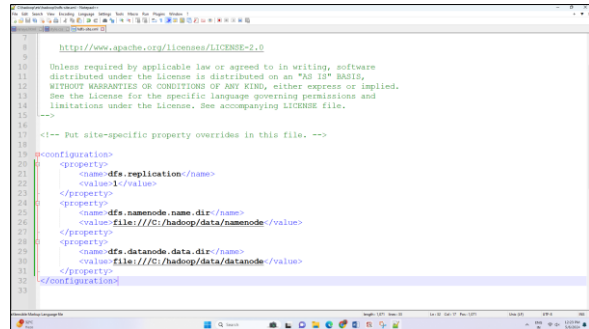
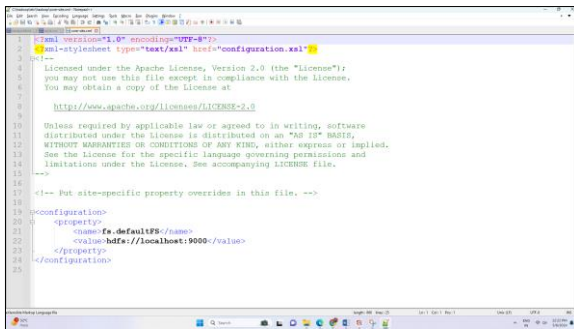
```
<configuration>
  <property>
    <name>dfs.replication</name>
    <value>1</value>
  </property>
  <property>
    <name>dfs.namenode.name.dir</name>
    <value>file:///C:/hadoop/data/namenode</value>
  </property>
  <property>
    <name>dfs.datanode.data.dir</name>
    <value>file:///C:/hadoop/data/datanode</value>
  </property>
</configuration>
```

c. Mapred-site.xml

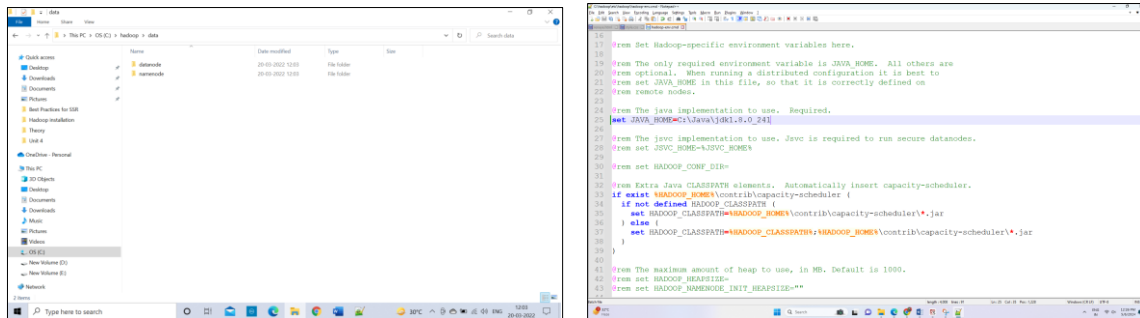
```
<configuration>
  <property>
    <name>mapred.framework.name</name>
    <value>yarn</value>
  </property>
</configuration>
```

d. Yarn-site.xml

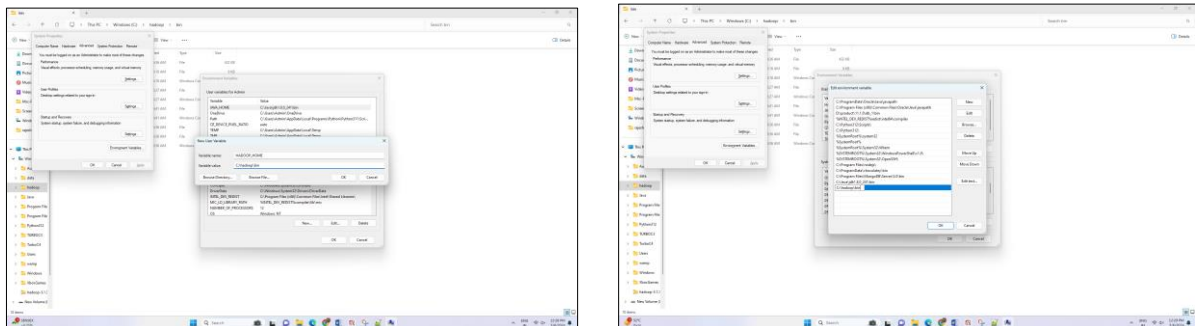
```
<configuration>
  <!-- Site specific YARN configuration properties -->
  <property>
    <name>yarn.nodemanager.aux-services</name>
    <value>mapreduce_shuffle</value>
  </property>
  <property>
    <name>yarn.nodemanager.auxservices.mapreduce.shuffle.class</name>
    <value>org.apache.hadoop.mapred.ShuffleHandler</value>
  </property>
</configuration>
```



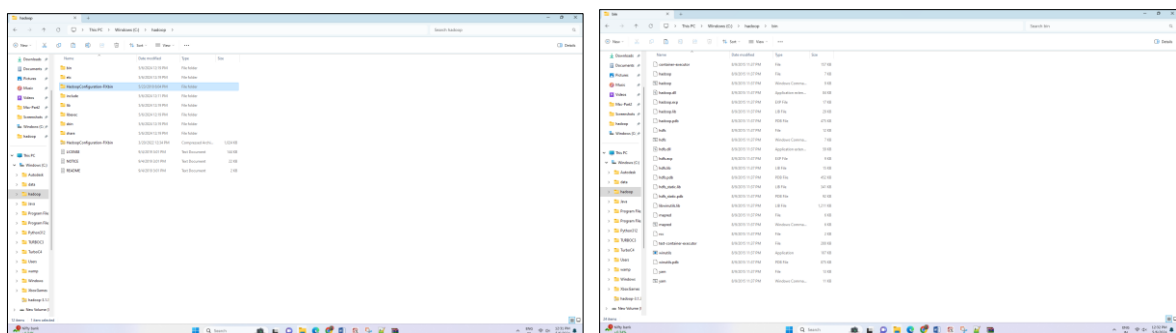
10. Create two directories “namenode” & “datanode” under data directory. And add java jdk directory path to “hadoop-env.cmd” file.



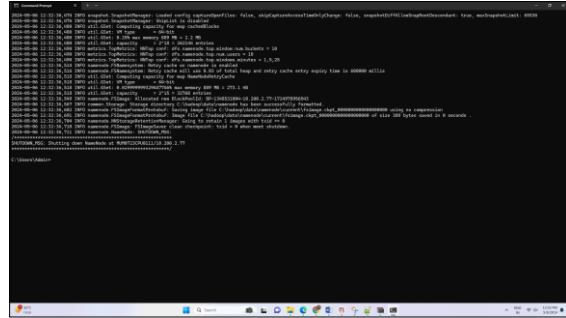
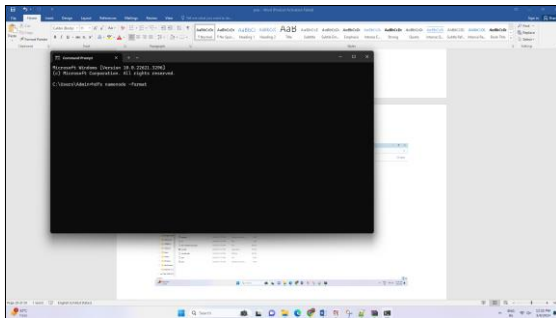
11. Set the environment variable and system path for hadoop.



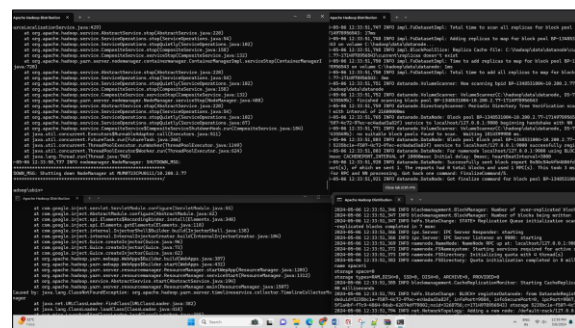
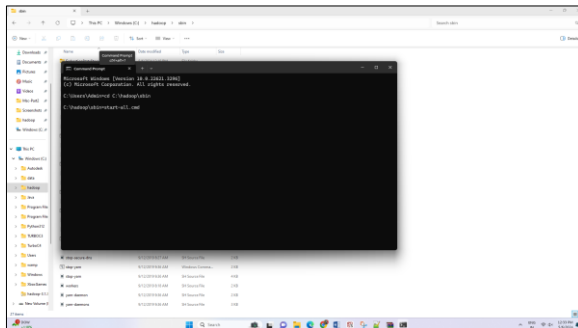
12. Download hadoop configuration file and extract it in the hadoop folder.



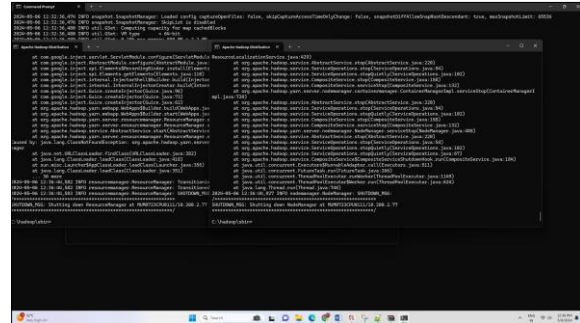
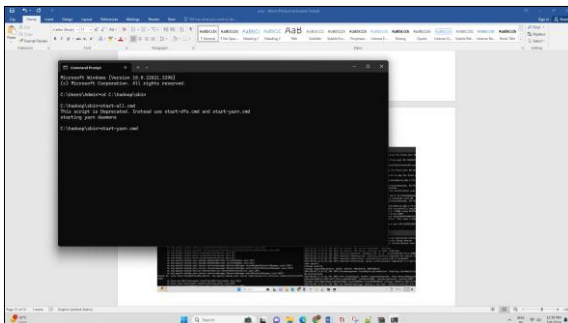
13. Once all the directory and system variables are set up. Test the hadoop installation. Open cmd and type “hdfs namenode -format”



14. Run “start-all.cmd” command to start all HDFS services.



15. Run “start-yarn.cmd” command to start all YARN services



Practical 2

Aim: Implement an application that stores big data in Hbase / MongoDB and manipulate it using R / Python.

Theory:

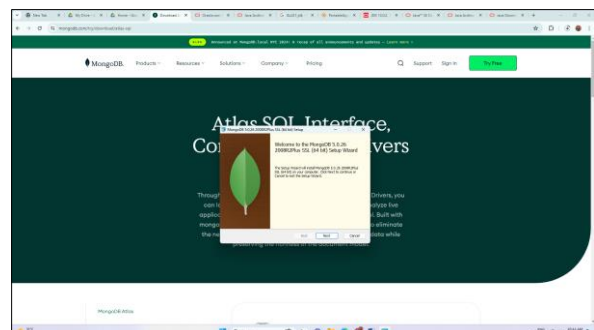
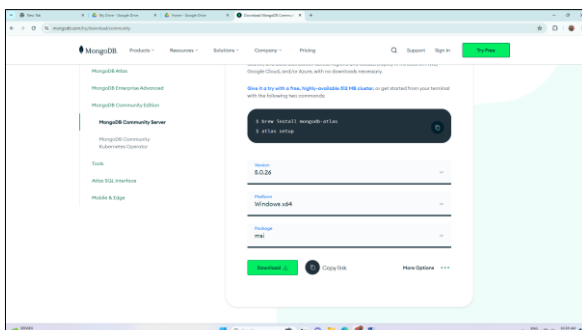
MongoDB is an open- source NoSQL database management program. NoSQL (Not only SQL) is used as an alternative to traditional relational databases. NoSQL databases are quite useful for working with large sets of distributed data. MongoDB is a tool that can manage document-oriented information, store or retrieve information. MongoDB is used for high-volume data storage, helping organizations store large amounts of data while still performing rapidly. Organizations also use MongoDB for its ad-hoc queries, indexing, load balancing, aggregation, server-side JavaScript execution and other features.

Features of MongoDB include the following:

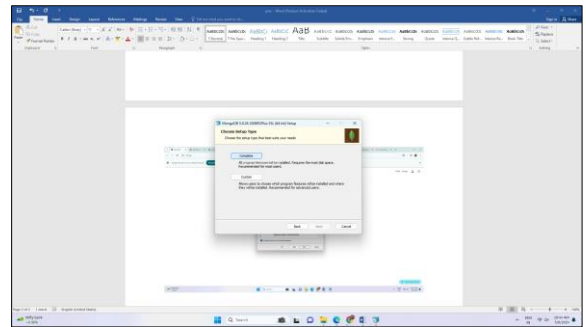
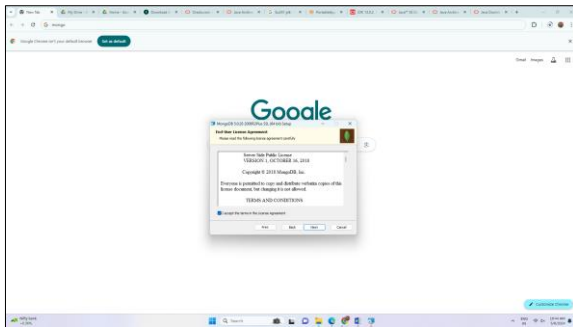
- **Replication:** A replica set is two or more MongoDB instances used to provide high availability. Replica sets are made of primary and secondary servers. The primary MongoDB server performs all the read and write operations, while the secondary replica keeps a copy of the data. If a primary replica fails, the secondary replica is then used.
- **Scalability:** MongoDB supports vertical and horizontal scaling. Vertical scaling works by adding more power to an existing machine, while horizontal scaling works by adding more machines to a user's resources.
- **Load balancing:** MongoDB handles load balancing without the need for a separate, dedicated load balancer, through either vertical or horizontal scaling.
- **Schema-less:** MongoDB is a schema-less database, which means the database can manage data without the need for a blueprint.
- **Document:** Data in MongoDB is stored in documents with key-value pairs instead of rows and columns, which makes the data more flexible when compared to SQL databases.

Steps: To download and install MongoDB:

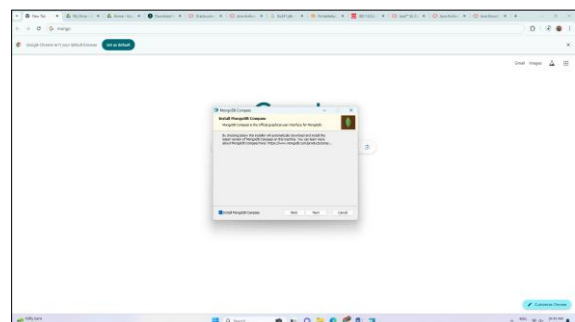
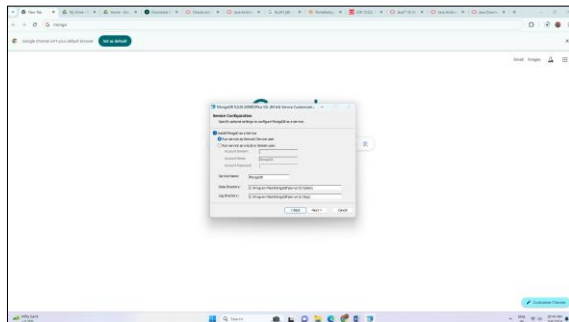
1. Download the 5.0.26 version of MongoDB double click on the executable file.



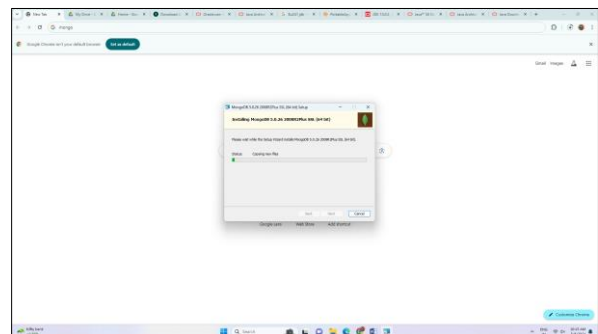
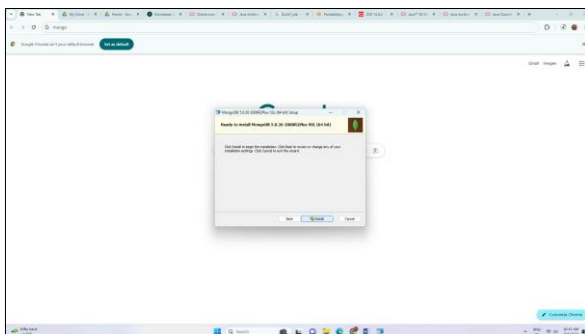
2. Accept all the license agreement and choose complete setup type from the setup wizard.



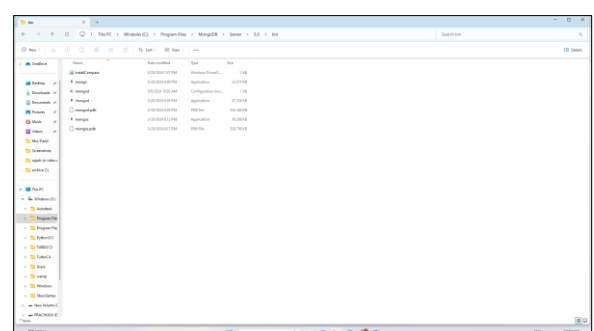
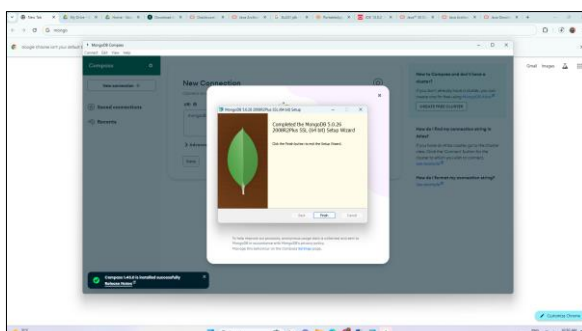
3. Service configuration dialog box appears click on next and its ready to install.



4. Click on install button to start installation.



5. Once the installation is completed click on finish and go to the bin folder of the installed file.



9. Once the mongo shell is started verify the different components using below commands.

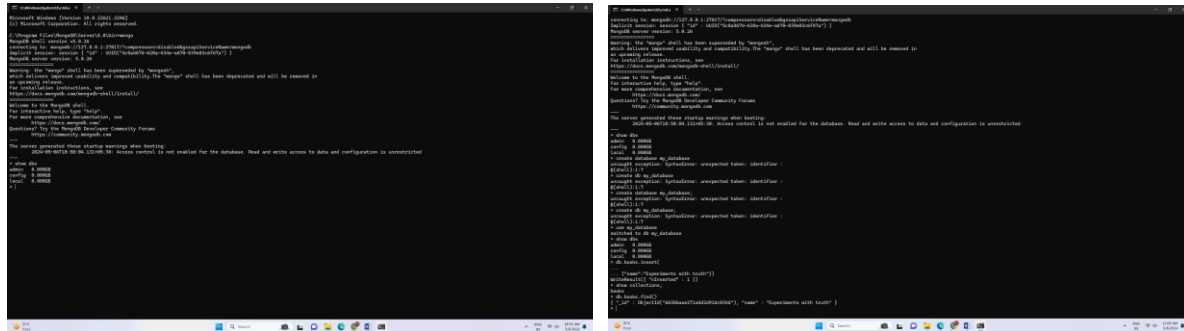
Commands: >show dbs -- To show dbs present in system.

>use my_database -- To create new db and use it.

>db.books.insert({"name": "Experiments with Truth"}) -- To insert json values in db.

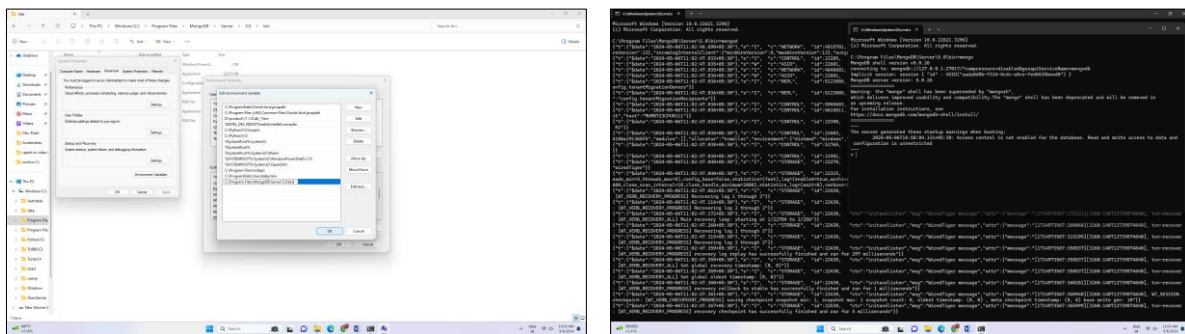
>show collections -- To see entered collections.

>db.books.find() – To see all collections in db.

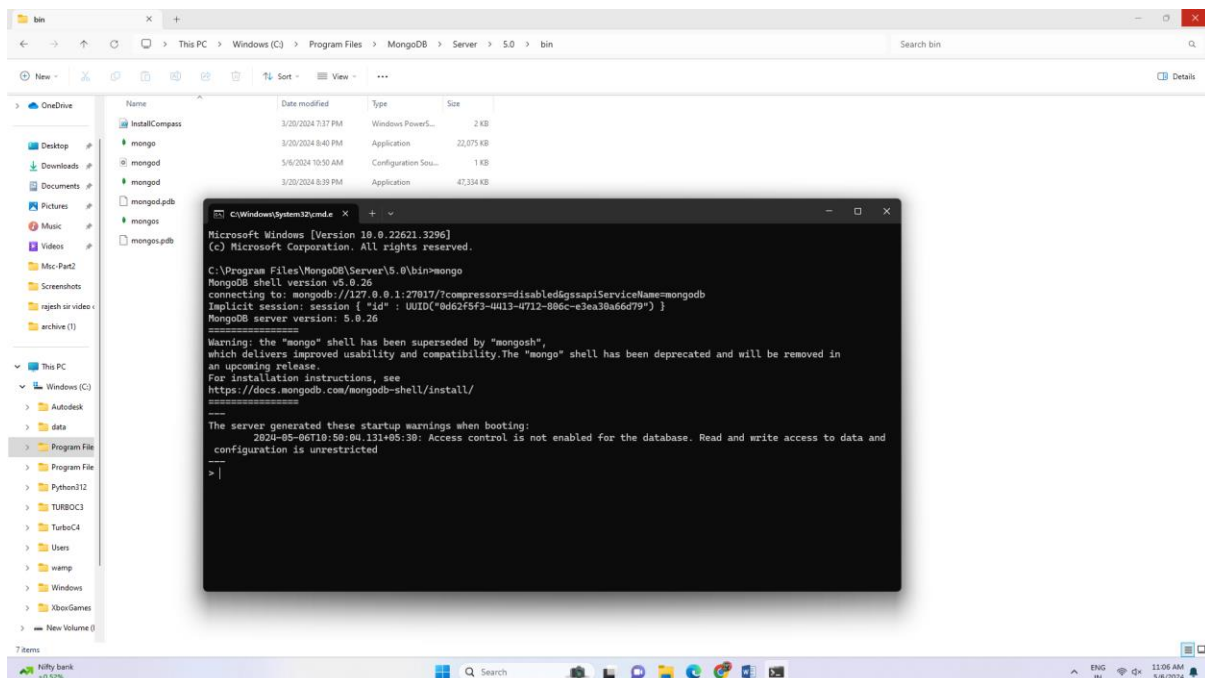


The first terminal screenshot shows the MongoDB shell version 5.0.26. The second terminal screenshot shows the output of the 'show dbs' command, listing the 'local' and 'config' databases. The third terminal screenshot shows the output of the 'use my_database' command, switching to the 'my_database' database. The fourth terminal screenshot shows the output of the 'db.books.insert' command, inserting a document into the 'books' collection. The fifth terminal screenshot shows the output of the 'show collections' command, listing the 'books' collection. The sixth terminal screenshot shows the output of the 'db.books.find()' command, displaying the inserted document.

10. Set system path for mongo db. And retry “mongod” and “mongo” command.

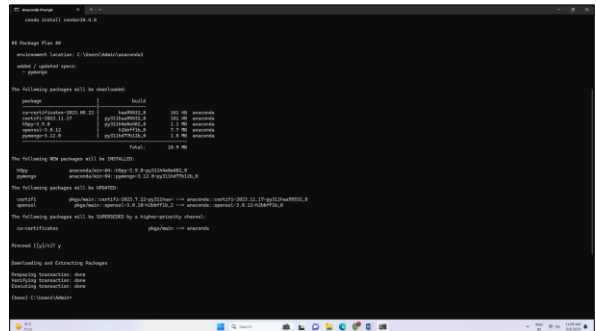
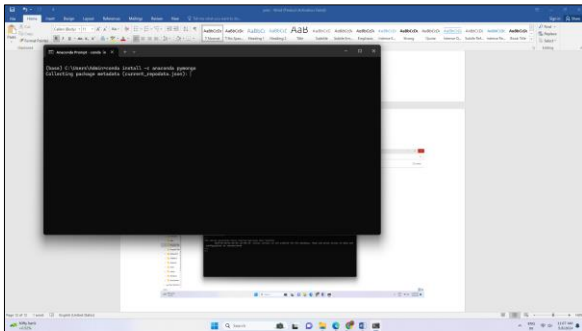


11. This should start the mongo demon and shell automatically.



12. To connect mongodb with python install below package.

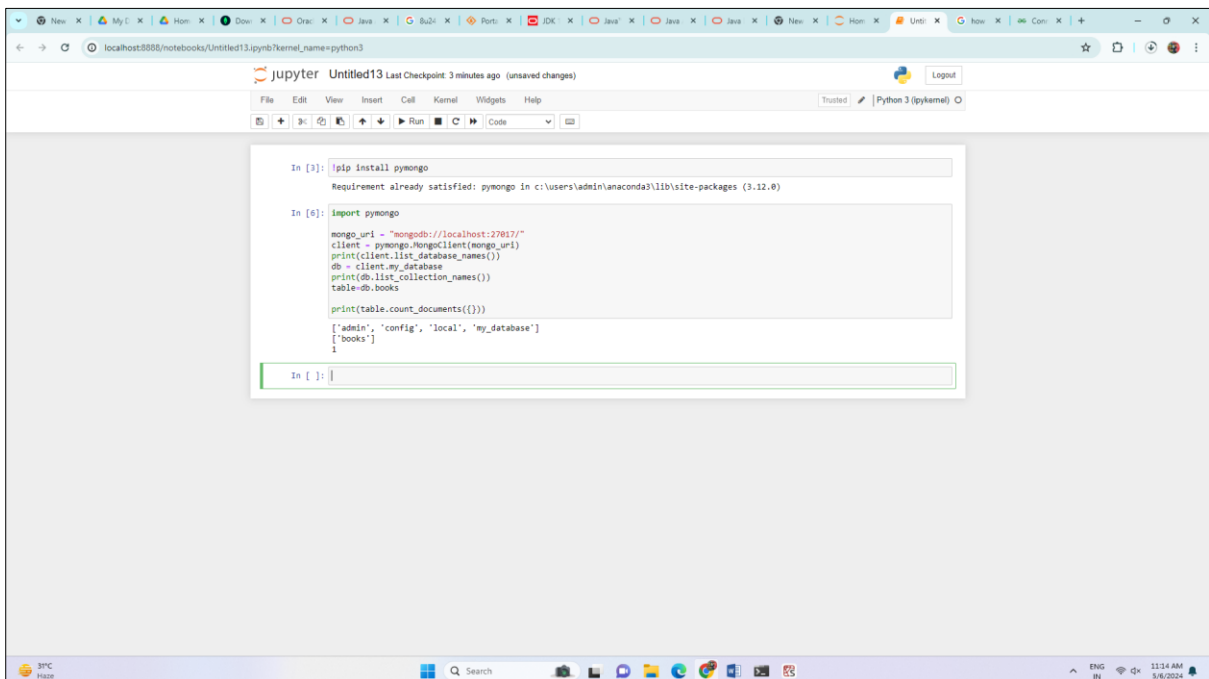
>conda install -c anaconda pymongo



13. Once the package is installed verify the connection

14. Run below code on python

```
import pymongo
mongo_uri = "mongodb://localhost:27017/"
client = pymongo.MongoClient(mongo_uri)
print(client.list_database_names())
db = client.my_database
print(db.list_collection_names())
table=db.books
```



Practical 3

Aim: Implement Regression Model Import a data from web storage. Name the dataset and now do Logistic Regression to find out relation between variables. Also check the model is fit or not.

Theory:

Linear Regression is a machine learning algorithm based on supervised learning. It performs a regression task. Regression models a target prediction value based on independent variables. It is mostly used for finding out the relationship between variables and forecasting. Different regression models differ based on – the kind of relationship between the dependent and independent variables, they are considering and the number of independent variables being used.

Code:

```
import matplotlib.pyplot as plt

import numpy as np

from sklearn import datasets, linear_model

from sklearn.metrics import mean_squared_error, r2_score

# Load the dataset

diabetes = datasets.load_diabetes()

# Load the diabetes dataset

diabetes_X, diabetes_y = datasets.load_diabetes(return_X_y=True)

# Description of the dataset

#print(diabetes['DESCR'])

# Use only one feature

diabetes_X = diabetes_X[:, np.newaxis, 2]

# Split the data into training/testing sets

diabetes_X_train = diabetes_X[:-20]

diabetes_X_test = diabetes_X[-20:]

# Split the targets into training/testing sets

diabetes_y_train = diabetes_y[:-20]

diabetes_y_test = diabetes_y[-20:]

# Create linear regression object

regr = linear_model.LinearRegression()

# Train the model using the training sets
```

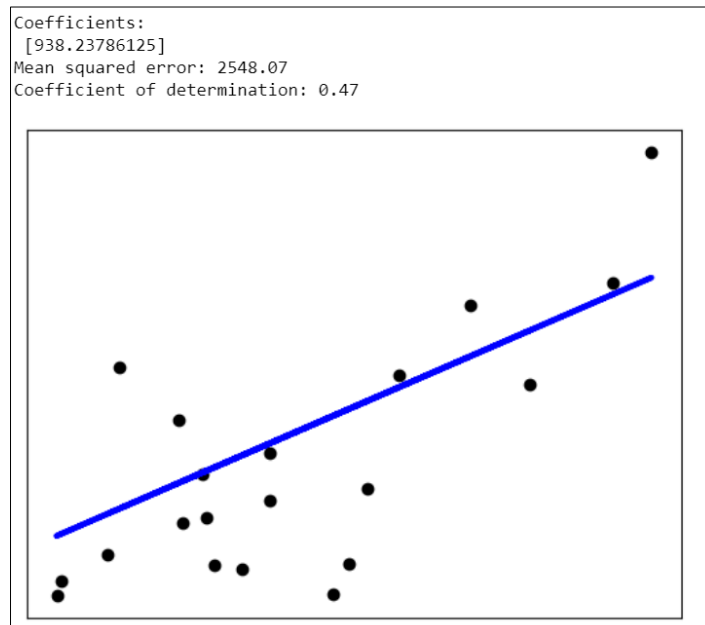
```

regr.fit(diabetes_X_train, diabetes_y_train)
# Make predictions using the testing set
diabetes_y_pred = regr.predict(diabetes_X_test)

# The coefficients
print("Coefficients: \n", regr.coef_)
# The mean squared error
print("Mean squared error: %.2f" % mean_squared_error(diabetes_y_test,
diabetes_y_pred))
# The coefficient of determination: 1 is perfect prediction
print("Coefficient of determination: %.2f" % r2_score(diabetes_y_test,
diabetes_y_pred))
# Plot outputs
plt.scatter(diabetes_X_test, diabetes_y_test, color="black")
plt.plot(diabetes_X_test, diabetes_y_pred, color="blue", linewidth=3)
plt.xticks(())
plt.yticks(())
plt.show()

```

Output:



Practical 4

Aim: Apply multiple regressions, if data have a continuous independent variable. Apply on above dataset.

Theory:

Multiple regression analysis is a statistical technique that analyzes the relationship between two or more variables and uses the information to estimate the value of the dependent variables. In multiple regression, the objective is to develop a model that describes a dependent variable y to more than one independent variable. Used to predict a continuous dependent variable based on multiple independent variables. Multiple linear regression should be used when there are multiple independent variables.

Multiple regression analysis permits to control explicitly for many other circumstances that concurrently influence the dependent variable. The objective of regression analysis is to model the relationship between a dependent variable and one or more independent variables.

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn import datasets, linear_model
from sklearn.metrics import mean_squared_error, r2_score
diabetes = datasets.load_diabetes()
diabetes_X, diabetes_y = datasets.load_diabetes(return_X_y=True)
# Description of the dataset
print(diabetes['DESCR'])
print(diabetes.feature_names)
diabetes_X = diabetes_X[:, np.newaxis, 0]
diabetes_X_train = diabetes_X[:-30]
diabetes_X_test = diabetes_X[-30:]
diabetes_y_train = diabetes_y[:-30]
diabetes_y_test = diabetes_y[-30:]
regr = linear_model.LinearRegression()
regr.fit(diabetes_X_train, diabetes_y_train)
diabetes_y_pred = regr.predict(diabetes_X_test)
print('Age')
print("Coefficients: \n", regr.coef_)
print("Mean squared error: %.2f" % mean_squared_error(diabetes_y_test,
diabetes_y_pred))
print("Coefficient of determination: %.2f" % r2_score(diabetes_y_test,
diabetes_y_pred))
plt.scatter(diabetes_X_test, diabetes_y_test, color="red")
plt.plot(diabetes_X_test, diabetes_y_pred, color="red", linewidth=2,
label='Age')
plt.xticks(())
plt.yticks(())
plt.title('Multiple Regression')
```

```

plt.xlabel('Age')
plt.ylabel('Disease Progression')
diabetes_X, diabetes_y = datasets.load_diabetes(return_X_y=True)
print(diabetes.feature_names)
diabetes_X = diabetes_X[:, np.newaxis, 3]
diabetes_X_train = diabetes_X[:-30]
diabetes_X_test = diabetes_X[-30:]
diabetes_y_train = diabetes_y[:-30]
diabetes_y_test = diabetes_y[-30:]
regr = linear_model.LinearRegression()
regr.fit(diabetes_X_train, diabetes_y_train)
# Make predictions using the testing set
diabetes_y_pred = regr.predict(diabetes_X_test)
print('BP')
print("Coefficients: \n", regr.coef_)
print("Mean squared error: %.2f" % mean_squared_error(diabetes_y_test,
diabetes_y_pred))
print("Coefficient of determination: %.2f" % r2_score(diabetes_y_test,
diabetes_y_pred))
plt.scatter(diabetes_X_test, diabetes_y_test, color="blue")
plt.plot(diabetes_X_test, diabetes_y_pred, color="blue", linewidth=2,
label='BP')
plt.xticks(())
plt.yticks(())
plt.title('Multiple Regression')
plt.ylabel('Disease Progression')
diabetes_X, diabetes_y = datasets.load_diabetes(return_X_y=True)
print(diabetes.feature_names)
diabetes_X = diabetes_X[:, np.newaxis, 2]
diabetes_X_train = diabetes_X[:-30]
diabetes_X_test = diabetes_X[-30:]
diabetes_y_train = diabetes_y[:-30]
diabetes_y_test = diabetes_y[-30:]
regr = linear_model.LinearRegression()
regr.fit(diabetes_X_train, diabetes_y_train)
diabetes_y_pred = regr.predict(diabetes_X_test)
print('BMI')
print("Coefficients: \n", regr.coef_)
print("Mean squared error: %.2f" % mean_squared_error(diabetes_y_test,
diabetes_y_pred))
print("Coefficient of determination: %.2f" % r2_score(diabetes_y_test,
diabetes_y_pred))
plt.scatter(diabetes_X_test, diabetes_y_test, color="black")
plt.plot(diabetes_X_test, diabetes_y_pred, color="black", linewidth=2,
label='BMI')
plt.xticks(())
plt.yticks(())
plt.title('Multiple Regression')
plt.ylabel('Disease Progression')
plt.legend() plt.show()

```

Output:

```
.. _diabetes_dataset:

Diabetes dataset
-----

Ten baseline variables, age, sex, body mass index, average blood
pressure, and six blood serum measurements were obtained for each of n =
442 diabetes patients, as well as the response of interest, a
quantitative measure of disease progression one year after baseline.

**Data Set Characteristics:**

:Number of Instances: 442

:Number of Attributes: First 10 columns are numeric predictive values

:Target: Column 11 is a quantitative measure of disease progression one year after baseline

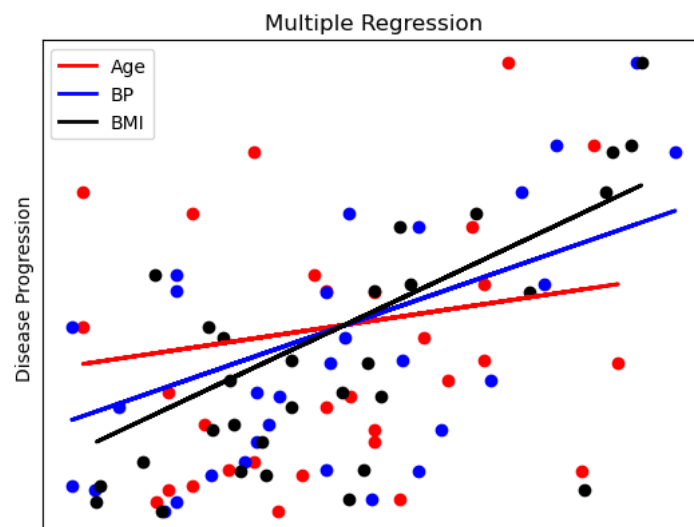
:Attribute Information:
- age      age in years
- sex
- bmi      body mass index
- bp       average blood pressure
- s1       tc, total serum cholesterol
- s2       ldl, low-density lipoproteins
- s3       hdl, high-density lipoproteins
- s4       tch, total cholesterol / HDL
- s5       ltg, possibly log of serum triglycerides level
- s6       glu, blood sugar level

Note: Each of these 10 feature variables have been mean centered and scaled by the standard deviation times the square root of
`n_samples` (i.e. the sum of squares of each column totals 1).

Source URL:
https://www4.stat.ncsu.edu/~boos/var.select/diabetes.html
```

For more information see:
Bradley Efron, Trevor Hastie, Iain Johnstone and Robert Tibshirani (2004) "Least Angle Regression," Annals of Statistics (with discussion), 407-499.
(https://web.stanford.edu/~hastie/Papers/LARS/LeastAngle_2002.pdf)

```
['age', 'sex', 'bmi', 'bp', 's1', 's2', 's3', 's4', 's5', 's6']
Age
Coefficients:
[298.46194553]
Mean squared error: 5275.14
Coefficient of determination: -0.02
['age', 'sex', 'bmi', 'bp', 's1', 's2', 's3', 's4', 's5', 's6']
BP
Coefficients:
[693.36333298]
Mean squared error: 3429.00
Coefficient of determination: 0.33
['age', 'sex', 'bmi', 'bp', 's1', 's2', 's3', 's4', 's5', 's6']
BMI
Coefficients:
[941.43097333]
Mean squared error: 3035.06
Coefficient of determination: 0.41
```



Practical 5

Aim: Build a classification model (Logistic Regression)

- Install relevant package for classification.
- Choose classifier for classification problem.
- Evaluate the performance of classifier.

Theory:

Logistic Regression is one of the simplest and commonly used Machine Learning algorithms for two-class classification. It is easy to implement and can be used as the baseline for any binary classification problem. Its basic fundamental concepts are also constructive in deep learning. Logistic regression describes and estimates the relationship between one dependent binary variable and independent variables.

It is a special case of linear regression where the target variable is categorical in nature. It uses a log of odds as the dependent variable. Logistic Regression predicts the probability of occurrence of a binary event utilizing a logit function.

Linear Regression Equation:

$$y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n$$

Where, y is a dependent variable and x1, x2 ... and Xn are explanatory variables.

Sigmoid Function:

$$p = \frac{1}{1 + e^{-y}}$$

Apply Sigmoid function on linear regression:

$$p = \frac{1}{1 + e^{-(\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n)}}$$

Code:

```
#import pandas
import pandas as pd
col_names = ['pregnant', 'glucose', 'bp', 'skin', 'insulin', 'bmi', 'pedigree', 'age', 'label']
# load dataset
pima = pd.read_csv("diabetes.csv", header=None, names=col_names)
pima.head()
#split dataset in features and target variable
feature_cols = ['pregnant', 'insulin', 'bmi', 'age', 'glucose', 'bp', 'pedigree']
X = pima[feature_cols] # Features
y = pima.label # Target variable
# split X and y into training and testing sets
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=16)
```

```

# import the class
from sklearn.linear_model import LogisticRegression
# instantiate the model (using the default parameters)
logreg = LogisticRegression(random_state=16)
# fit the model with data
logreg.fit(X_train, y_train)
y_pred = logreg.predict(X_test)
# import the metrics class
from sklearn import metrics
cnf_matrix = metrics.confusion_matrix(y_test, y_pred)
cnf_matrix

```

Output:

```

array([[116,   9],
       [ 26,  41]], dtype=int64)

```

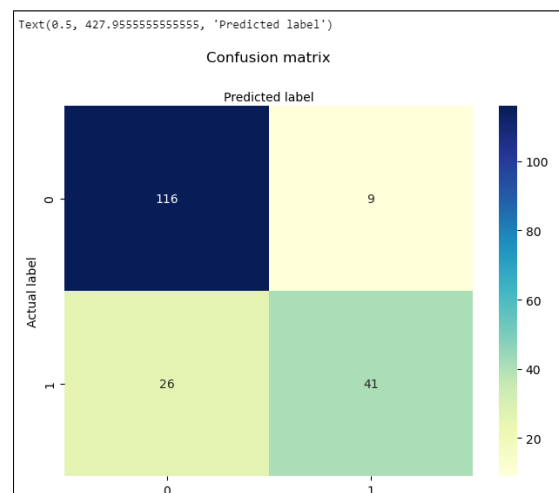
Code:

```

# import required modules
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
class_names=[0,1] # name of classes
fig, ax = plt.subplots()
tick_marks = np.arange(len(class_names))
plt.xticks(tick_marks, class_names)
plt.yticks(tick_marks, class_names)
# create heatmap
sns.heatmap(pd.DataFrame(cnf_matrix), annot=True, cmap="YlGnBu", fmt='g')
ax.xaxis.set_label_position("top")
plt.tight_layout()
plt.title('Confusion matrix', y=1.1)
plt.ylabel('Actual label')
plt.xlabel('Predicted label')

```

Output:



Code:

```
from sklearn.metrics import classification_report  
target_names = ['without diabetes', 'with diabetes']  
print(classification_report(y_test, y_pred, target_names=target_names))
```

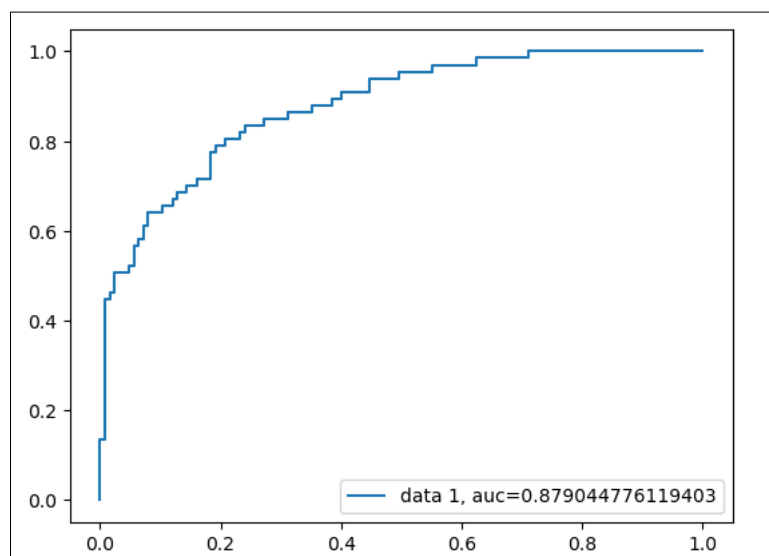
Output:

	precision	recall	f1-score	support
without diabetes	0.82	0.93	0.87	125
with diabetes	0.82	0.61	0.70	67
accuracy			0.82	192
macro avg	0.82	0.77	0.78	192
weighted avg	0.82	0.82	0.81	192

Code:

```
y_pred_proba = logreg.predict_proba(X_test)[::,1]  
fpr, tpr, _ = metrics.roc_curve(y_test, y_pred_proba)  
auc = metrics.roc_auc_score(y_test, y_pred_proba)  
plt.plot(fpr,tpr,label="data 1, auc="+str(auc))  
plt.legend(loc=4)  
plt.show()
```

Output:



Practical 6

Aim: Build a Clustering Model.

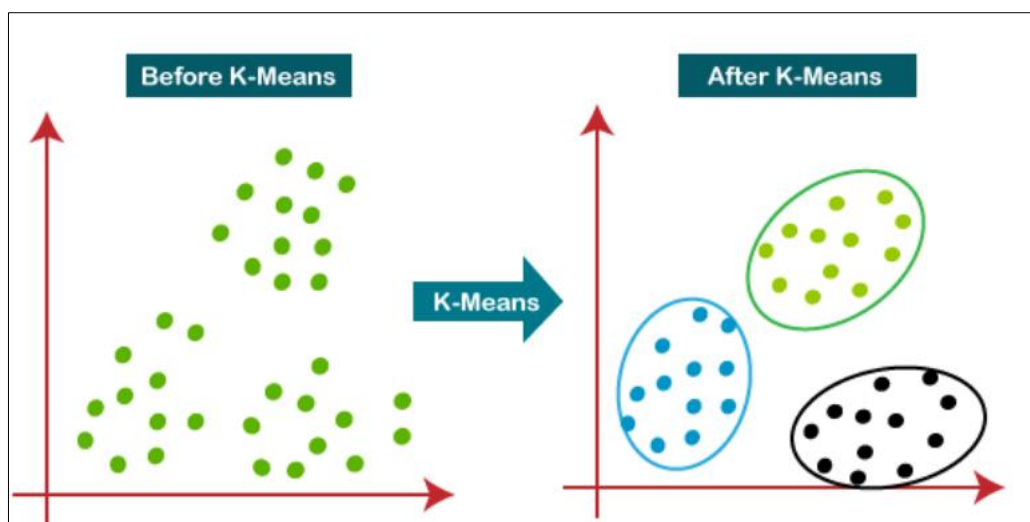
- Select clustering algorithm for unsupervised learning.
- Plot the cluster data using R/python visualizations.

Theory:

K-Means Clustering is an Unsupervised Learning algorithm, which groups the unlabeled dataset into different clusters. Here K defines the number of pre-defined clusters that need to be created in the process, as if $K=2$, there will be two clusters. It allows us to cluster the data into different groups and a convenient way to discover the categories of groups in the unlabeled dataset on its own without the need for any training.

It is a centroid-based algorithm, where each cluster is associated with a centroid. The main aim of this algorithm is to minimize the sum of distances between the data point and their corresponding clusters.

The algorithm takes the unlabeled dataset as input, divides the dataset into k-number of clusters, and repeats the process until it does not find the best clusters. The value of k should be predetermined in this algorithm.



Code:

```
# k-means clustering
from numpy import unique
from numpy import where
from sklearn.datasets import make_classification
from sklearn.cluster import KMeans
from matplotlib import pyplot

# define dataset
X, _ = make_classification(n_samples=1000, n_features=2, n_informative=2,
n_redundant=0, n_clusters_per_class=1, random_state=4)
```

```

# define the model
model = KMeans(n_clusters=2)

# fit the model
model.fit(X)

# assign a cluster to each example
yhat = model.predict(X)

# retrieve unique clusters
clusters = unique(yhat)

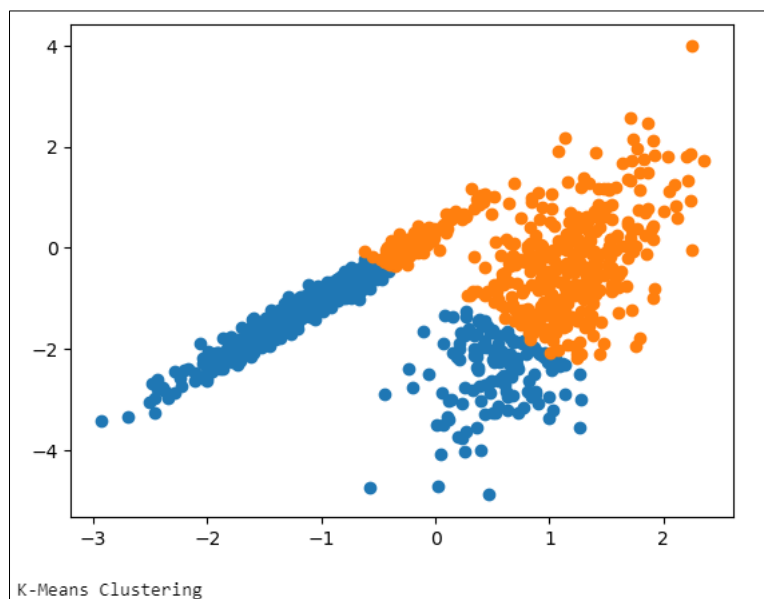
# create scatter plot for samples from each cluster
for cluster in clusters:
    # get row indexes for samples with this cluster
    row_ix = where(yhat == cluster)

    # create scatter of these samples
    pyplot.scatter(X[row_ix, 0], X[row_ix, 1])

# show the plot
pyplot.show()
print("K-Means Clustering")

```

Output:



Agglomerative Clustering

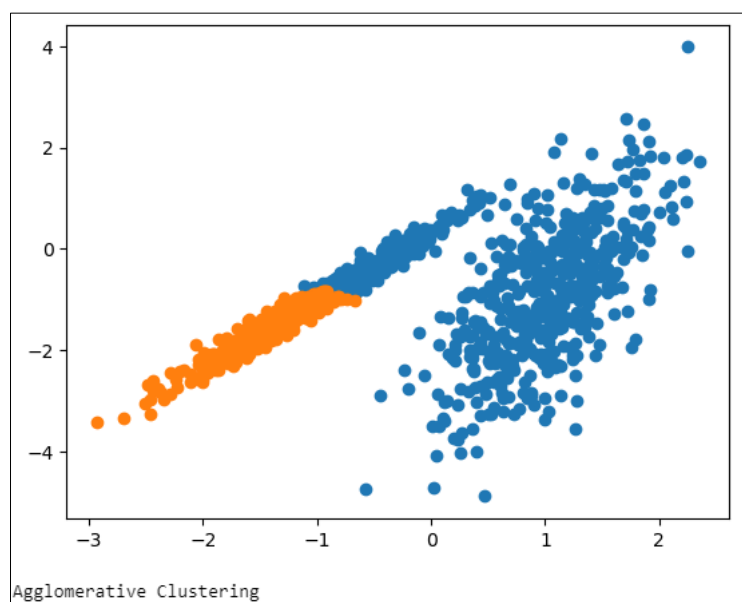
Theory:

The agglomerative hierarchical clustering algorithm is a popular example of HCA. To group the datasets into clusters, it follows the bottom-up approach. It means, this algorithm considers each dataset as a single cluster at the beginning, and then start combining the closest pair of clusters together. It does this until all the clusters are merged into a single cluster that contains all the datasets. This hierarchy of clusters is represented in the form of the dendrogram.

Code:

```
from numpy import where
from sklearn.datasets import make_classification
from sklearn.cluster import AgglomerativeClustering
from matplotlib import pyplot
X, _ = make_classification(n_samples=1000, n_features=2, n_informative=2, n_redundant=0,
n_clusters_per_class=1, random_state=4)
model = AgglomerativeClustering(n_clusters=2)
yhat = model.fit_predict(X)
clusters = unique(yhat)
# create scatter plot for samples from each cluster
for cluster in clusters:
    # get row indexes for samples with this cluster
    row_ix = where(yhat == cluster)
    # create scatter of these samples
    pyplot.scatter(X[row_ix, 0], X[row_ix, 1])
# show the plot
pyplot.show()
print("Agglomerative Clustering")
```

Output:



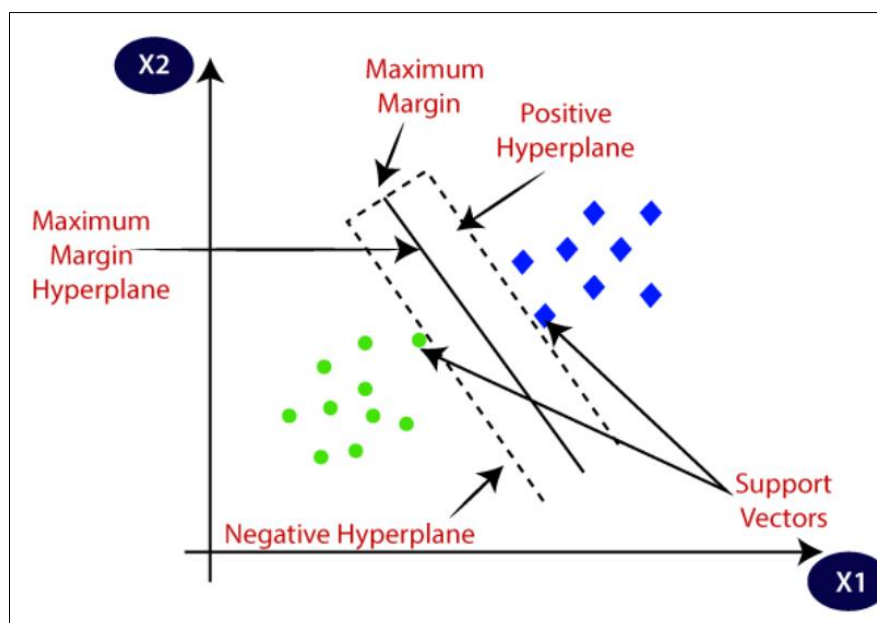
Practical 7

Aim: Implement SVM classification technique

Theory:

Support Vector Machine or SVM is one of the most popular Supervised Learning algorithms, which is used for Classification as well as Regression problems. However, primarily, it is used for Classification problems in Machine Learning.

The goal of the SVM algorithm is to create the best line or decision boundary that can segregate n-dimensional space into classes so that we can easily put the new data point in the correct category in the future. This best decision boundary is called a hyperplane. SVM chooses the extreme points/vectors that help in creating the hyperplane. These extreme cases are called as support vectors, and hence algorithm is termed as Support Vector Machine. Consider the below diagram in which there are two different categories that are classified using a decision boundary or hyperplane:



Code:

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
dataset = pd.read_csv('Social_Network_Ads.csv')
# Split Dataset into X and Y
X = dataset.iloc[:, [2, 3]].values
y = dataset.iloc[:, 4].values
#Split the X and Y Dataset into the Training set and Test set
```

```

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.25, random_state = 0)
#Perform Feature Scaling - Feature scaling helps us to normalize the data within a particular
range.
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
#Fit SVM to the Training set
from sklearn.svm import SVC
classifier = SVC(kernel = 'rbf', random_state = 0)
classifier.fit(X_train, y_train)
#Predict the Test Set Results
y_pred = classifier.predict(X_test)
#Make the Confusion Matrix
from sklearn.metrics import confusion_matrix, accuracy_score
cm = confusion_matrix(y_test, y_pred)
print(cm)
accuracy_score(y_test,y_pred)
#Visualise the Test set results
from matplotlib.colors import ListedColormap
X_set, y_set = X_test, y_test
X1, X2 = np.meshgrid(np.arange(start = X_set[:, 0].min() - 1, stop = X_set[:, 0].max() + 1,
step = 0.01),
np.arange(start = X_set[:, 1].min() - 1, stop = X_set[:, 1].max() + 1, step = 0.01))
plt.contourf(X1, X2, classifier.predict(np.array([X1.ravel(),
X2.ravel()]).T).reshape(X1.shape),
alpha = 0.75, cmap = ListedColormap(('red', 'green')))
plt.xlim(X1.min(), X1.max())
plt.ylim(X2.min(), X2.max())
for i, j in enumerate(np.unique(y_set)):

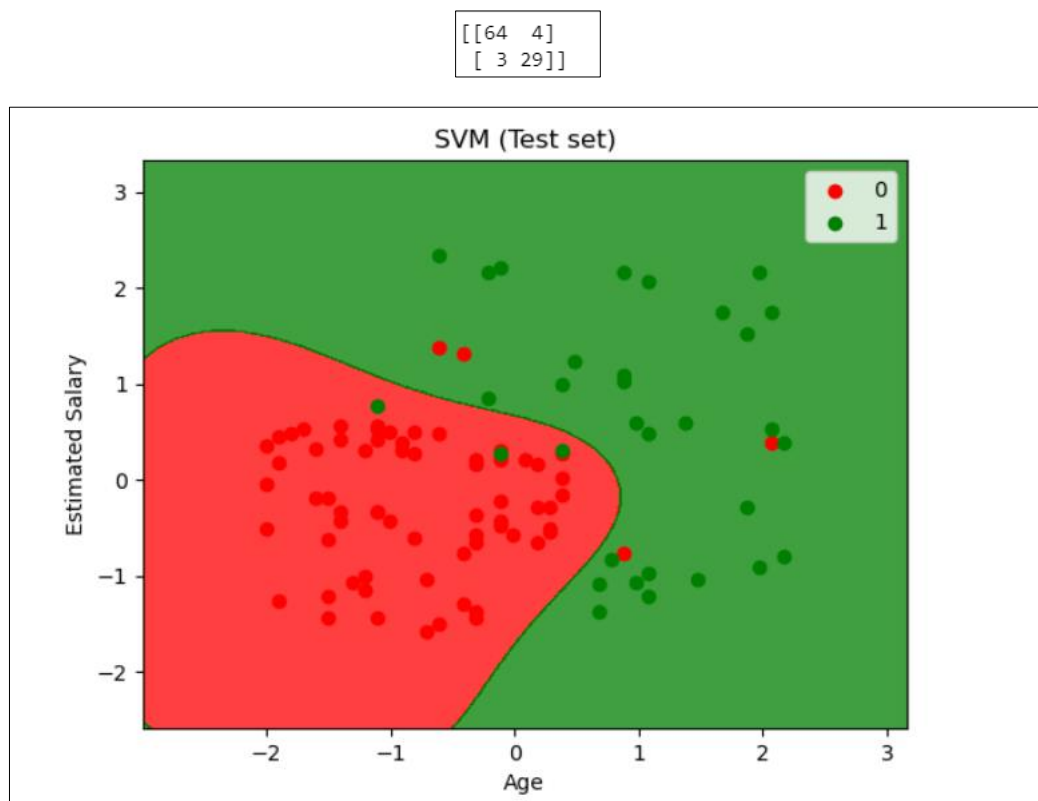
```

```

plt.scatter(X_set[y_set == j, 0], X_set[y_set == j, 1], c = ListedColormap(('red', 'green'))(i),
label = j)
plt.title('SVM (Test set)')
plt.xlabel('Age')
plt.ylabel('Estimated Salary')
plt.legend()
plt.show()

```

Output:



Practical 8

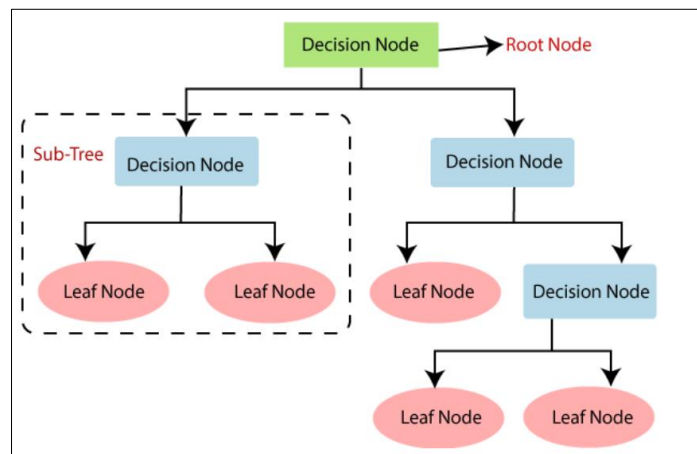
Aim: Implement Decision Tree classification technique.

Theory:

Decision Tree is a Supervised learning technique that can be used for both classification and Regression problems, but mostly it is preferred for solving Classification problems. It is a tree-structured classifier, where internal nodes represent the features of a dataset, branches represent the decision rules and each leaf node represents the outcome.

In a Decision tree, there are two nodes, which are the Decision Node and Leaf Node. Decision nodes are used to make any decision and have multiple branches, whereas Leaf nodes are the output of those decisions and do not contain any further branches. The decisions or the test are performed on the basis of features of the given dataset. It is a graphical representation for getting all the possible solutions to a problem/decision based on given conditions.

It is called a decision tree because, similar to a tree, it starts with the root node, which expands on further branches and constructs a tree-like structure.



Install below packages before running code:

`pip install pydotplus`

`conda install python-graphviz`

Code:

```
# Load libraries
import pandas as pd
from sklearn.tree import DecisionTreeClassifier # Import Decision Tree Classifier
from sklearn.model_selection import train_test_split # Import train_test_split function
from sklearn import metrics #Import scikit-learn metrics module for accuracy calculation
from io import StringIO

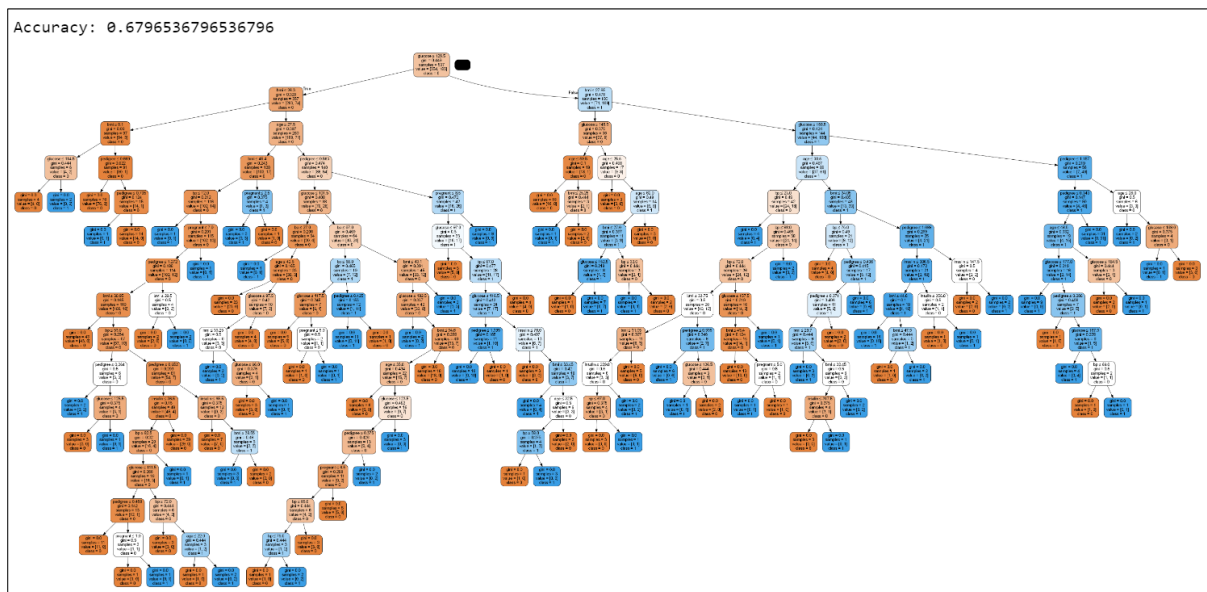
col_names = ['pregnant', 'glucose', 'bp', 'skin', 'insulin', 'bmi', 'pedigree', 'age', 'label']
# load dataset
pima = pd.read_csv("diabetes.csv", header=None, names=col_names)
pima.head()
#split dataset in features and target variable
```

```

feature_cols = ['pregnant', 'insulin', 'bmi', 'age', 'glucose', 'bp', 'pedigree']
X = pima[feature_cols] # Features
y = pima.label # Target variable
# Split dataset into training set and test set
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1) # 70%
training and 30% test
# Create Decision Tree classifier object
clf = DecisionTreeClassifier()
# Train Decision Tree Classifier
clf = clf.fit(X_train, y_train)
# Predict the response for test dataset
y_pred = clf.predict(X_test)
# Model Accuracy, how often is the classifier correct?
print("Accuracy:", metrics.accuracy_score(y_test, y_pred))
import six
# import sys
# sys.modules['sklearn.externals.six'] = six
# from sklearn.tree import export_graphviz
from six import StringIO
from IPython.display import Image
import pydotplus
dot_data = StringIO()
export_graphviz(clf, out_file=dot_data,
                filled=True, rounded=True,
                special_characters=True, feature_names = feature_cols, class_names=['0','1'])
graph = pydotplus.graph_from_dot_data(dot_data.getvalue())
graph.write_png('diabetes.png')
Image(graph.create_png())

```

Output:



Practical 9

Aim: - Naïve Bayes Implementation

Theory:

Naive Bayes algorithm is a supervised learning algorithm, which is based on Bayes theorem and used for solving classification problems.

The Naïve Bayes algorithm is comprised of two words Naïve and Bayes, which can be described as:

- **Naïve:** It is called Naïve because it assumes that the occurrence of a certain feature is independent of the occurrence of other features. Such as if the fruit is identified on the bases of color, shape, and taste, then red, spherical, and sweet fruit is recognized as an apple. Hence each feature individually contributes to identify that it is an apple without depending on each other.
- **Bayes:** It is called Bayes because it depends on the principle of Bayes' Theorem.

Bayes' theorem is also known as Bayes' Rule or Bayes' law, which is used to determine the probability of a hypothesis with prior knowledge. It depends on the conditional probability. The formula for Bayes' theorem is given as:

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

where,

$P(A|B)$ is *Posterior probability*: Probability of hypothesis A on the observed event B.

$P(B|A)$ is *Likelihood probability*: Probability of the evidence given that the probability of a hypothesis is true.

$P(A)$ is *Prior Probability*: Probability of hypothesis before observing the evidence.

$P(B)$ is *Marginal Probability*: Probability of Evidence.

Code:

```
# Importing the libraries
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
# Importing the dataset
dataset = pd.read_csv('D:/MSCIT_books/sem2/BDA_Pracs/Social_Network_Ads.csv')
dataset.head()
X = dataset.iloc[:, [2, 3]].values
y = dataset.iloc[:, 4].values
X = dataset.iloc[:, [2, 3]].values
y = dataset.iloc[:, 4].values
# Splitting the dataset into the Training set and Test set
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.25, random_state = 0)
# Feature Scaling
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
```

```

X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
# Fitting Naive Bayes to the Training set
from sklearn.naive_bayes import GaussianNB
classifier = GaussianNB()
classifier.fit(X_train, y_train)
# Predicting the Test set results
y_pred = classifier.predict(X_test)
y_pred
# Making the Confusion Matrix
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred)
# Visualising the Training set results
from matplotlib.colors import ListedColormap
X_set, y_set = X_train, y_train
X1, X2 = np.meshgrid(np.arange(start = X_set[:, 0].min() - 1, stop = X_set[:, 0].max() + 1,
step = 0.01),
np.arange(start = X_set[:, 1].min() - 1, stop = X_set[:, 1].max() + 1, step =
0.01))
plt.contourf(X1, X2, classifier.predict(np.array([X1.ravel(),
X2.ravel()]).T).reshape(X1.shape),
alpha = 0.75, cmap = ListedColormap(('red', 'green')))
plt.xlim(X1.min(), X1.max())
plt.ylim(X2.min(), X2.max())
for i, j in enumerate(np.unique(y_set)):
plt.scatter(X_set[y_set == j, 0], X_set[y_set == j, 1],
c = ListedColormap(('red', 'green'))(i), label = j)
plt.title('Naive Bayes (Training set)')
plt.xlabel('Age')
plt.ylabel('Estimated Salary')
plt.legend()
plt.show()
# Visualising the Test set results
from matplotlib.colors import ListedColormap
X_set, y_set = X_test, y_test
X1, X2 = np.meshgrid(np.arange(start = X_set[:, 0].min() - 1, stop = X_set[:, 0].max() + 1,
step = 0.01),
np.arange(start = X_set[:, 1].min() - 1, stop = X_set[:, 1].max() + 1, step =
0.01))
plt.contourf(X1, X2, classifier.predict(np.array([X1.ravel(),
X2.ravel()]).T).reshape(X1.shape),
alpha = 0.75, cmap = ListedColormap(('red', 'green')))
plt.xlim(X1.min(), X1.max())
plt.ylim(X2.min(), X2.max())
for i, j in enumerate(np.unique(y_set)):
plt.scatter(X_set[y_set == j, 0], X_set[y_set == j, 1],

```

```

c = ListedColormap(('red', 'green'))(i, label = j)
plt.title('Naive Bayes (Test set)')
plt.xlabel('Age')
plt.ylabel('Estimated Salary')
plt.legend()
plt.show()

```

Output:

